KIỂM THỬ PHẦN MỀM

TÀI LIỆU THỰC HÀNH

LAB 11 - CI/CD + SELENIUM GRID + CHIẾN LƯỢC KIỂM THỬ DỰ ÁN THỰC TẾ

GitHub Actions | Docker + Selenium Grid | Test Strategy | Chương trình 7 bài tập

Nội dung	Chi tiết
Chủ đề	GitHub Actions CI/CD, Selenium Grid 4, Docker, Test Strategy, Test Plan
Kỹ năng đạt được	Vận hành pipeline test tự động, chạy cross-browser, lập kế hoạch kiểm thử
Công cụ	GitHub Actions, Docker Desktop, Selenium Grid 4, Allure Commandline
Thời gian	240 phút | Điểm số: 10 điểm
Điều kiện tiên quyết	Có tài khoản GitHub. Hoàn thành Lab 9 (framework POM chuẩn)
Cài đặt cần thiết	Docker Desktop, Git, GitHub CLI (gh), Allure Commandline

Mục tiêu bài lab

Sau khi hoàn thành Lab 11, sinh viên có thể:

- ✓ Viết GitHub Actions workflow chạy Selenium test tự động khi push code
- ✓ Cấu hình matrix strategy để chạy song song nhiều browser cùng lúc
- ✓ Khởi động Selenium Grid bằng Docker và kết nối framework với Grid
- ✓ Đo và so sánh hiệu quả khi chạy test tuần tự vs song song trên Grid
- ✓ Xử lý bảo mật: lưu credential vào GitHub Secrets, không commit vào code
- ✓ Soạn Test Strategy Document chuyên nghiệp cho dự án phần mềm thực tế
- ✓ Viết Test Plan cho một Sprint cụ thể với test case ưu tiên rõ ràng

I. TÓM TẮT LÝ THUYẾT

Các trang web tham khảo để soạn bài lab 11:
- https://viblo.asia/p/nghien-cuu-ve-grid-trong-selenium-automation-testing-MLzkOLRBvpq
- https://topdev.vn/blog/thuc-thi-kiem-thu-tu-dong-selenium-voi-selenium-grid/
- https://anhtester.com/lesson/selenium-java-bai-3-gioi-thieu-selenium-webdriver
- https://tech.cybozu.vn/huong-dan-cai-dat-selenium-grid-3-tren-windows-32b17/
- https://viblo.asia/p/huong-dan-ve-docker-selenium-tutorial-cach-tich-hop-selenium-grid-voi-docker-oOVlYWvVK8W
- https://tech.cybozu.vn/huong-dan-cai-dat-selenium-grid-bang-ky-thuat-docker-da699/
- https://github.com/SeleniumHQ/docker-selenium
- https://talent500.com/blog/docker-integration-with-selenium-grid/
- https://www.automatedtestingwithtuyen.com/post/part-2-set-up-selenium-grid-with-docker-using-different-machines

➢ Anh chị có thể tham khảo bằng cách nhấp vào các đường link ở trên sẽ thấy các nội dung lý thuyết liên quan!

11.1 CI/CD và vai trò của kiểm thử tự động trong pipeline

CI (Continuous Integration) là thực hành developer push code thường xuyên (nhiều lần mỗi ngày) vào một nhánh chung. CD (Continuous Deployment/Delivery) là tự động deploy ứng dụng lên môi trường sau khi code pass các bước kiểm tra.

Quy trình phát triển có CI/CD trong thực tế

1. Developer viết code, commit và push lên nhánh feature/login-fix
2. Developer tạo Pull Request (PR) vào nhánh main
3. GitHub Actions tự động kích hoạt → chạy toàn bộ test suite (smoke + regression)
4. Nếu test FAIL → PR bị block, không merge được → developer phải sửa
5. Nếu test PASS → code reviewer xem xét, approve và merge

6. Sau khi merge vào main → pipeline chạy lại + deploy lên staging tự động
7. Hàng đêm 2AM → pipeline chạy full regression test để kiểm tra tích lũy

Kết quả: Lỗi được phát hiện trong vòng 10 phút sau khi developer push code.

✓ Không có CI/CD: Lỗi được phát hiện 2 tuần sau khi merge, khi đã release đến khách hàng.

Thành phần CI/CD	Công cụ phổ biến	Vai trò trong kiểm thử
Source Control	Git + GitHub/GitLab/Bitbucket	Trigger pipeline khi push/PR
CI Server	GitHub Actions, Jenkins, CircleCI	Chạy test tự động, báo cáo kết quả
Test Runner	Maven + TestNG, Gradle + JUnit	Thực thi test, xuất báo cáo XML
Test Report	Allure, ExtentReports, Surefire	Hiển thị kết quả đẹp cho team xem
Coverage Tool	JaCoCo	Đo % code được test, enforce minimum
Secret Manager	GitHub Secrets, Vault	Lưu password/API key an toàn
Notification	Slack, Email, Teams	Thông báo khi pipeline fail

11.2 GitHub Actions - Cấu trúc và cú pháp

GitHub Actions dùng file YAML trong thư mục .github/workflows/ của repository. Mỗi file là một workflow:

```
# .github/workflows/selenium-tests.yml

name: Selenium Test Suite   # Tên hiển thị trên GitHub UI

# === TRIGGER: Khi nào pipeline chạy ===
on:
  push:
    branches: [ main, develop ]     # Chạy khi push lên main hoặc
develop
  pull_request:
    branches: [ main ]              # Chạy khi mở PR nhắm vào main
```

```yaml
  schedule:
    - cron: '0 2 * * 1-5'          # 2:00 AM mỗi ngày thứ 2-6 (UTC)
  workflow_dispatch:  # Nút "Run workflow" trên GitHub UI - dùng khi cần chạy thủ công

# === JOBS: Các công việc cần làm ===
jobs:
  run-tests:
    runs-on: ubuntu-latest # Máy ảo Linux do GitHub cung cấp miễn phí

    # === MATRIX: Chạy song song nhiều browser ===
    strategy:
      matrix:
        browser: [chrome, firefox]
      fail-fast: false          # Chrome fail không dừng Firefox

    steps:
      # Bước 1: Lấy source code
      - name: Checkout code
        uses: actions/checkout@v4

      # Bước 2: Cài Java 17
      - name: Set up Java 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven    # Cache .m2 để lần sau download nhanh hơn

      # Bước 3: Chạy test
      - name: Run Selenium Tests (${{ matrix.browser }})
        run: |
          mvn clean test \
            -Dbrowser=${{ matrix.browser }} \
            -Denv=staging \
            -DsuiteXmlFile=testng-smoke.xml
        env:
          # Đọc từ GitHub Secrets - KHÔNG hardcode password
          APP_PASSWORD: ${{ secrets.SAUCEDEMO_PASSWORD }}

      # Bước 4: Lưu kết quả test (chạy DÙ test fail hay pass)
      - name: Upload test results
        if: always()   # always() = chạy kể cả khi bước trước fail
        uses: actions/upload-artifact@v4
        with:
          name: test-results-${{ matrix.browser }}
          path: |
```

```
            target/surefire-reports/
            target/allure-results/
            target/screenshots/
          retention-days: 30       # Giữ artifact 30 ngày
```

11.3 Headless Browser - Bắt buộc khi chạy trên CI

Máy chủ CI (GitHub Actions, Jenkins...) không có màn hình thật. Nếu không bật Headless mode, test sẽ fail với lỗi "cannot open display":

```
// DriverFactory.java - Hỗ trợ Headless tự động trên CI
public class DriverFactory {
    public static WebDriver createDriver(String browser) {
        // Tự động detect môi trường CI
        boolean isCI = System.getenv("CI") != null;  // GitHub
Actions set CI=true tự động
        return switch (browser.toLowerCase()) {
            case "firefox" -> createFirefoxDriver(isCI);
            default        -> createChromeDriver(isCI);
        };
    }
    private static WebDriver createChromeDriver(boolean headless) {
        WebDriverManager.chromedriver().setup();
        ChromeOptions options = new ChromeOptions();
        if (headless) {
            options.addArguments("--headless=new");      // Chrome
112+ headless mới
            options.addArguments("--no-sandbox");        // BẮT
BUỘC trên Linux CI
            options.addArguments("--disable-dev-shm-usage"); //
Tránh OOM trên container
            options.addArguments("--window-size=1920,1080"); // Set
resolution cố định
            System.out.println("[Driver] Chạy Chrome HEADLESS (CI
mode)");
```

```
        } else {
            options.addArguments("--start-maximized");
            System.out.println("[Driver] Chạy Chrome bình thường
(Local mode)");
        }
        return new ChromeDriver(options);
    }
    private static WebDriver createFirefoxDriver(boolean headless)
{
        WebDriverManager.firefoxdriver().setup();
        FirefoxOptions options = new FirefoxOptions();
        if (headless) {
            options.addArguments("-headless");
        }
        return new FirefoxDriver(options);
    }
}
```

11.4 Allure Report trong CI/CD - Báo cáo tự động publish

Allure tạo báo cáo HTML trực quan, hiển thị biểu đồ, lịch sử chạy, ảnh chụp khi fail. Trong CI/CD, báo cáo được publish tự động lên GitHub Pages:

```
<!-- pom.xml: Thêm Allure -->
<properties>
    <allure.version>2.26.0</allure.version>
</properties>
<dependencies>
    <dependency>
        <groupId>io.qameta.allure</groupId>
        <artifactId>allure-testng</artifactId>
        <version>${allure.version}</version>
    </dependency>
```

```
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>io.qameta.allure</groupId>
            <artifactId>allure-maven</artifactId>
            <version>2.12.0</version>
        </plugin>
    </plugins>
</build>
// Annotations trong test class:
@Test
@Feature("Đăng nhập hệ thống")              // Nhóm theo tính năng
@Story("UC-001: Đăng nhập bằng tài khoản hợp lệ")  // User story
@Description("Kiểm thử đăng nhập với username/password hợp lệ")
@Severity(SeverityLevel.CRITICAL)            // Mức độ nghiêm trọng
public void testLoginSuccess() {
    Allure.step("Mở trang đăng nhập", () -> driver.get(BASE_URL));
    Allure.step("Nhập thông tin đăng nhập", () -> {
        loginPage.enterUsername("standard_user");
        loginPage.enterPassword("secret_sauce");
    });
    Allure.step("Click nút Đăng nhập", () ->
loginPage.clickLoginButton());
    Allure.step("Kiểm tra chuyển trang thành công", () ->

Assert.assertTrue(driver.getCurrentUrl().contains("inventory")));
}
// Đính kèm ảnh chụp vào Allure khi fail — trong @AfterMethod của
BaseTest:
@Attachment(value = "Screenshot", type = "image/png")
public byte[] attachScreenshot(WebDriver driver) {
```

```
    return ((TakesScreenshot)
driver).getScreenshotAs(OutputType.BYTES);
}
```

11.5 Selenium Grid 4 - Kiến trúc và cách hoạt động

Selenium Grid cho phép phân phối việc chạy test ra nhiều máy (node) khác nhau. Grid 4 hỗ trợ Docker natively, dễ setup hơn nhiều so với Grid 3:

Kiến trúc Selenium Grid 4

Hub (Router): Là điểm trung tâm nhận request từ test code.

→ Nhận RemoteWebDriver request, tìm node phù hợp (browser/OS đúng)

→ Phân phối session đến node thích hợp

→ Grid UI: http://localhost:4444 - xem trạng thái node, session đang chạy

Node: Máy thực sự chạy browser.

→ Đăng ký với Hub, báo cáo: tôi có Chrome x64, Firefox x64, tối đa 3 session

→ Chạy WebDriver session, stream video kết quả về Hub

Ưu điểm so với chạy local:

- Chạy song song thực sự trên nhiều máy khác nhau
- Kiểm thử cross-browser (Chrome + Firefox + Edge + Safari) cùng lúc
- Không bị giới hạn bởi CPU/RAM của máy developer
- Phù hợp cho Regression test cần chạy 500+ test trong vài phút

```
# docker-compose.yml - Selenium Grid với Docker
version: '3.8'
services:
  # Hub: Điều phối trung tâm
  selenium-hub:
```

```
  image: selenium/hub:4.18.1
  container_name: selenium-hub
  ports:
    - "4442:4442"    # Event bus
    - "4443:4443"    # Event bus
    - "4444:4444"    # Grid UI và WebDriver endpoint
# Chrome Node 1: Tối đa 3 session đồng thời
chrome-node-1:
  image: selenium/node-chrome:4.18.1
  shm_size: 2gb          # Quan trọng: Chrome cần shared memory
  depends_on:
    - selenium-hub
  environment:
    - SE_EVENT_BUS_HOST=selenium-hub
    - SE_NODE_MAX_SESSIONS=3
    - SE_NODE_SESSION_TIMEOUT=300

# Chrome Node 2
chrome-node-2:
  image: selenium/node-chrome:4.18.1
  shm_size: 2gb
  depends_on:
    - selenium-hub
  environment:
    - SE_EVENT_BUS_HOST=selenium-hub
    - SE_NODE_MAX_SESSIONS=3
# Firefox Node
```

```
  firefox-node:
    image: selenium/node-firefox:4.18.1
    shm_size: 2gb
    depends_on:
      - selenium-hub
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_NODE_MAX_SESSIONS=2
# Lệnh khởi động Grid:
# docker-compose up -d
# Xem trạng thái: http://localhost:4444
# Dừng Grid: docker-compose down
```

Sửa BaseTest/DriverFactory để kết nối với Grid:

```
// DriverFactory.java - Hỗ trợ cả Local và Grid (Remote)
public static WebDriver createDriver(String browser) {
    String gridUrl = System.getProperty("grid.url");

    if (gridUrl != null && !gridUrl.isBlank()) {
        // Chạy trên Selenium Grid
        return createRemoteDriver(browser, gridUrl);
    }
    // Chạy local
    return createLocalDriver(browser);
}
private static WebDriver createRemoteDriver(String browser, String gridUrl) {
    DesiredCapabilities caps = new DesiredCapabilities();
    caps.setBrowserName(browser.toLowerCase());
```

```
    // Grid 4: Hỗ trợ Chrome Options qua W3C
    if (browser.equalsIgnoreCase("chrome")) {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--no-sandbox", "--disable-dev-shm-usage");
        caps.merge(options);
    }
    try {
        URL gridEndpoint = new URL(gridUrl + "/wd/hub");
        RemoteWebDriver driver = new RemoteWebDriver(gridEndpoint, caps);
        // Tăng timeout khi dùng Grid (mạng có latency)
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        System.out.println("[Grid] Session tạo thành công trên: " + gridUrl
                        + " | Browser: " + browser
                        + " | SessionID: " + driver.getSessionId());
        return driver;
    } catch (MalformedURLException e) {
        throw new RuntimeException("Grid URL không hợp lệ: " + gridUrl);
    }
}
// Chạy với Grid cục bộ:
// mvn test -Dgrid.url=http://localhost:4444 -Dbrowser=chrome -Dparallel=true
// Chạy local (không Grid):
// mvn test -Dbrowser=chrome
```

11.6 GitHub Secrets - Bảo mật credential trong CI/CD

Một trong những lỗi bảo mật phổ biến nhất: hardcode mật khẩu vào code rồi push lên GitHub. Dữ liệu nhạy cảm phải được lưu trữ trong GitHub Secrets:

Loại thông tin	Lưu ở đâu	Không bao giờ lưu ở đâu

Mật khẩu test	GitHub Secrets (Settings → Secrets)	config.properties, Java code, README
API Key	GitHub Secrets	Bất kỳ file nào commit vào git
Database URL staging	GitHub Secrets	docker-compose.yml committed vào git
URL công khai	config-staging.properties	-
Browser, timeout	config.properties	-
Test description	Test method, @Test(description)	-

```
# Cách đặt GitHub Secrets:
# 1. Mở repository trên GitHub
# 2. Settings → Secrets and variables → Actions → New repository secret
# 3. Name: SAUCEDEMO_PASSWORD | Value: secret_sauce
# Sử dụng trong workflow YAML:
- name: Run tests
  run: mvn test -Dbrowser=chrome
  env:
    APP_PASSWORD: ${{ secrets.SAUCEDEMO_PASSWORD }}   # Inject vào biến
môi trường
    APP_USERNAME: ${{ secrets.SAUCEDEMO_USERNAME }}
# Đọc trong Java (ConfigReader hoặc BaseTest):
// Đọc biến môi trường - ưu tiên hơn file config
String password = System.getenv("APP_PASSWORD");
if (password == null || password.isBlank()) {
    // Fallback: đọc từ config (môi trường local)
```

```
    password = ConfigReader.getInstance().getProperty("app.password");
}
# Kiểm tra: GitHub sẽ tự động mask giá trị Secret trong log
# Log sẽ hiển thị: APP_PASSWORD=*** (không lộ giá trị thật)
```

11.7 Test Strategy - Tài liệu kiểm thử cấp dự án

Test Strategy là tài liệu cấp cao mô tả "cách tiếp cận kiểm thử" của toàn dự án. Được QA Lead soạn thảo ở đầu dự án, cập nhật khi có thay đổi lớn:

Phần của Test Strategy	Mục đích	Nội dung ví dụ
1. Phạm vi kiểm thử	Xác định rõ IN SCOPE và OUT SCOPE	IN: Thanh toán, đăng nhập | OUT: Admin dashboard (phase 2)
2. Loại kiểm thử	Quyết định test gì, bao nhiêu %	70% API test, 20% UI test, 10% E2E regression
3. Môi trường	Mô tả từng environment	Dev: localhost | Staging: mirror production | Prod: dùng synthetic monitoring
4. Công cụ và framework	Liệt kê + lý do chọn	Selenium + POM (vì team biết Java, POM dễ bảo trì)
5. Entry/Exit Criteria	Khi nào bắt đầu/dừng test	Entry: dev done, code deploy lên staging | Exit: 95% pass, 0 P1 bug
6. Rủi ro kiểm thử	Rủi ro kỹ thuật và nghiệp vụ	Staging data xóa thường xuyên, API 3rd party không ổn định
7. Lịch trình	Khi nào chạy loại test nào	Mỗi commit: smoke test | Hàng đêm: regression | Trước release: UAT

8. Quản lý lỗi	Quy trình báo cáo và theo dõi	Jira: P1 fix trong 24h, P2 trong 1 sprint, P3 backlog

11.8 Test Plan cho Sprint - Lập kế hoạch kiểm thử ngắn hạn

Test Plan khác Test Strategy ở chỗ: Test Strategy là tài liệu toàn dự án (hàng tháng), Test Plan là tài liệu cho một Sprint/release cụ thể (từ 2 tuần trở lên):

Mục trong Test Plan	Câu hỏi cần trả lời	Ví dụ thực tế Sprint 3
Sprint Goal	Tính năng nào cần test trong sprint này?	Thanh toán bằng ví điện tử MoMo
Test Scope	Test case nào cần viết mới? Cái nào dùng lại?	15 TC mới: API payment + 5 TC UI + 8 TC regression cũ
Test Schedule	Bao giờ test? Bao giờ xong?	Dev done: ngày 8/sprint | Test start: ngày 9 | Done: ngày 13
Resources	Ai test cái gì?	QA 1: API test | QA 2: UI test | Dev tự unit test
Rủi ro của sprint	Điều gì có thể chặn công việc kiểm thử?	API MoMo sandbox không ổn định tuần trước
Định nghĩa Done	Khi nào tính năng được coi là đã test xong?	95% TC pass, 0 bug P1/P2, Allure report xanh, coverage >= 80%

II. BÀI TẬP THỰC HÀNH

Bài 1 **CẤU HÌNH GITHUB ACTIONS CƠ BẢN (1.5 điểm)**

Tạo repository GitHub và thiết lập pipeline CI/CD đầu tiên chạy Selenium test tự động:

❖ **Bước chuẩn bị:**

1. Push framework từ Lab 9 lên repository GitHub mới tên selenium-framework
2. Thêm README.md với mô tả ngắn về project và hướng dẫn chạy local
3. Tạo .gitignore phù hợp: bỏ qua target/, .idea/, *.iml, screenshots/

❖ **Cấu hình pipeline (0.75 điểm):**

- Tạo .github/workflows/selenium-ci.yml với trigger: push vào main, pull_request, workflow_dispatch
- Cài Java 17 (Temurin), bật Maven cache để lần sau build nhanh hơn
- Chạy mvn test với tham số browser=chrome, env=dev, suiteXmlFile=testng-smoke.xml
- Bật Headless mode: Đọc biến môi trường CI trong DriverFactory, tự động thêm --headless=new cho Chrome
- Upload artifact: lưu target/surefire-reports/ và target/screenshots/ khi test fail (if: failure())

❖ **Kiểm chứng (0.75 điểm):**

- Push code lên main → Actions tab hiển thị pipeline đang chạy → chờ kết quả
- Chụp màn hình: Actions log hiển thị màu xanh ✓ (tất cả test pass)
- Tạo 1 test sai assertion cố ý → push → pipeline fail → chụp màn hình log đỏ ×
- Download artifact → xem ảnh chụp màn hình của test bị fail được lưu trong artifact

Lỗi thường gặp khi chạy Selenium trên GitHub Actions

Lỗi 1: 'Chrome not found' → Giải pháp: Ubuntu Latest đã có Chrome sẵn. Dùng WebDriverManager tự động match version

Lỗi 2: 'cannot open display :0' → Bạn quên bật --headless, xem lại DriverFactory

Lỗi 3: 'Out of memory' → Thêm --disable-dev-shm-usage vào ChromeOptions

Lỗi 4: 'Element not interactable' → Tăng explicit wait. CI chậm hơn local máy

Lỗi 5: Test pass local nhưng fail CI → Thường do timing. Kiểm tra lại implicit/explicit wait

Bài 2	MATRIX STRATEGY - CHẠY SONG SONG ĐA BROWSER (1 điểm)

Cấu hình pipeline chạy song song Chrome và Firefox cùng lúc, tối ưu thời gian CI:

- Thêm strategy.matrix vào workflow, matrix.browser: [chrome, firefox], fail-fast: false
- Sửa DriverFactory đã hỗ trợ Firefox headless: options.addArguments("-headless")
- Đặt tên artifact theo browser: test-results-${{ matrix.browser }} - 2 artifact riêng cho 2 browser
- Chạy pipeline → Chụp màn hình Actions tab cho thấy 2 job chạy song song (Chrome job và Firefox job cùng lúc)
- So sánh thời gian: Chạy tuần tự (tắt matrix, browser=chrome;firefox) vs song song (matrix) - ghi lại kết quả

```
# Mẫu cấu hình
# testng-smoke.xml - Cấu hình nhận tham số browser từ Maven/CI
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Smoke Test Suite" verbose="1">
  <!-- browser được truyền từ -Dbrowser= khi chạy Maven -->
  <parameter name="browser" value="chrome"/>
  <parameter name="env" value="dev"/>
  <test name="Login Smoke Tests">
    <groups>
      <run>
        <include name="smoke"/>
      </run>
    </groups>
    <classes>
```

```
      <class name="tests.LoginTest"/>
    </classes>
  </test>
</suite>
```

Bài 3 GITHUB SECRETS - BẢO MẬT CREDENTIAL (1 điểm)

Bảo mật credential là yêu cầu bắt buộc trong mọi dự án thực tế. Bài này đảm bảo không có password nào bị commit vào repository:

4. Vào Settings → Secrets and variables → Actions → Tạo 2 secret:

SAUCEDEMO_USERNAME với giá trị standard_user

SAUCEDEMO_PASSWORD với giá trị secret_sauce

5. Sửa workflow YAML: Truyền secret qua env block
6. Sửa ConfigReader để ưu tiên đọc System.getenv("SAUCEDEMO_PASSWORD") trước, fallback về config file nếu không có
7. Kiểm tra bảo mật: Thêm step "echo $APP_PASSWORD" vào workflow → GitHub sẽ hiển thị *** thay vì giá trị thật
8. Tạo .env.example (không commit .env) - file mẫu cho developer local biết cần đặt biến gì

❖ **Tiêu chí bắt buộc**

• Tìm kiếm trong toàn bộ codebase: grep -r 'secret_sauce' src/ → kết quả RỖNG (không có hardcode)

• Tìm kiếm: grep -r 'standard_user' src/main/ → kết quả RỖNG

• Pipeline chạy thành công dù credential chỉ nằm trong GitHub Secrets

Bài 4 SELENIUM GRID VỚI DOCKER (2 điểm)

Khởi động Selenium Grid cục bộ bằng Docker và chạy test phân tán thực sự:

Phần A - Khởi động Grid (0.5 điểm):

- Tạo docker-compose.yml với 1 Hub, 2 Chrome Node (mỗi node max 3 session), 1 Firefox Node
- Chạy: docker-compose up -d - xác nhận tất cả container running: docker ps
- Mở http://localhost:4444 → Grid Console UI → chụp màn hình cho thấy 3 node đã đăng ký với Hub

Phần B - Kết nối framework với Grid (0.75 điểm):

- Sửa DriverFactory theo mẫu ở phần lý thuyết: khi có -Dgrid.url → dùng RemoteWebDriver
- Tạo testng-grid.xml: parallel="tests", thread-count="4", 4 <test> section chạy cùng lúc
- Chạy: mvn test -Dgrid.url=http://localhost:4444 -DsuiteXmlFile=testng-grid.xml
- Trong lúc chạy, mở Grid Console UI → chụp màn hình cho thấy nhiều session đang hoạt động đồng thời

Phần C - Đo hiệu suất (0.75 điểm):

- Chuẩn bị 8 test method (có thể dùng từ Lab 1-2)
- Chạy 1: Tuần tự (thread-count=1) - ghi lại thời gian chạy tổng (T1)
- Chạy 2: Song song 4 thread trên Grid - ghi lại thời gian (T2)
- Tính hệ số tăng tốc = T1 / T2. Điền vào bảng sau và nộp:

Cấu hình	Số thread	Thời gian chạy	Hệ số tăng tốc
Tuần tự (local)	1	... giây	1.0x (baseline)
Song song (Grid 2 thread)	2	... giây	... x
Song song (Grid 4 thread)	4	... giây	... x

Bài 5 ALLURE REPORT NÂNG CAO VỚI ANNOTATION (1 điểm)

Nâng cấp bộ test từ Lab 9 để tạo Allure Report chuyên nghiệp như trong dự án thực:

- Thêm Allure dependency vào pom.xml và cấu hình allure-maven plugin

- Thêm @Feature, @Story, @Severity, @Description cho tất cả @Test method trong LoginTest, CartTest
- Thêm Allure.step() bên trong mỗi test để ghi lại từng bước: "Mở trang login", "Nhập username", "Click đăng nhập", "Xác nhận chuyển trang"
- Triển khai ScreenshotOnFailureListener: khi test FAIL → gọi Allure.addAttachment() đính kèm ảnh PNG
- Chạy: mvn clean test, sau đó: mvn allure:serve → Báo cáo mở tự động trong browser
- Chụp màn hình Allure Report cho thấy: biểu đồ pass/fail, phân loại theo Feature, step-by-step, ảnh khi fail

Bài 6 PIPELINE ĐẦY ĐỦ VỚI ALLURE PUBLISH LÊN GITHUB PAGES (1.5 điểm)

Nâng cấp workflow để tự động publish Allure Report lên GitHub Pages sau mỗi lần test chạy thành công:

```
# .github/workflows/selenium-full.yml - Pipeline hoàn chỉnh
name: Full Selenium CI Pipeline

on:
  push:
    branches: [ main ]
  schedule:
    - cron: '0 2 * * 1-5'   # 2AM mỗi ngày trong tuần

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        browser: [chrome, firefox]
      fail-fast: false

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin', cache:
maven }

      - name: Run tests
```

```
        run: mvn clean test -Dbrowser=${{ matrix.browser }} -
DsuiteXmlFile=testng-smoke.xml
        env:
          APP_USERNAME: ${{ secrets.SAUCEDEMO_USERNAME }}
          APP_PASSWORD: ${{ secrets.SAUCEDEMO_PASSWORD }}

      - name: Upload Allure results
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: allure-results-${{ matrix.browser }}
          path: target/allure-results/

  publish-report:
    needs: test           # Chạy sau khi tất cả job test xong
    if: always()          # Chạy kể cả khi test fail
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Download Chrome results
        uses: actions/download-artifact@v4
        with: { name: allure-results-chrome, path: allure-results/
}

      - name: Download Firefox results
        uses: actions/download-artifact@v4
        with: { name: allure-results-firefox, path: allure-results/
}

      - name: Generate & Publish Allure Report
        uses: simple-elf/allure-report-action@master
        with:
          allure_results: allure-results
          gh_pages: gh-pages
          allure_report: allure-report
```

- Bật GitHub Pages: Settings → Pages → Source: Deploy from a branch → Branch: gh-pages
- Sau khi pipeline chạy xong → Allure Report xuất hiện tại https://{username}.github.io/{repo}/
- Chụp màn hình link GitHub Pages đang hoạt động với Allure Report đầy đủ

- Thêm badge vào README.md: [![Test Status](badge-url)] và [![Allure Report](pages-url)]

Bài 7	SOẠN TEST STRATEGY VÀ TEST PLAN CHO DỰ ÁN THỰC TẾ (2 điểm)

Bài tập tổng hợp cuối cùng - Bạn đóng vai QA Lead của startup ShopEasy. Đây là kỹ năng phân biệt QA junior và senior, và là điều nhà tuyển dụng thực sự kiểm tra khi phỏng vấn.

Giả sử có Bối cảnh dự án ShopEasy

- Dự án: Ứng dụng mua sắm online - ShopEasy
- Tech stack: Java Spring Boot (API) + React (Web) + React Native (Mobile)
- Team: 4 Developer (2 backend, 2 frontend), 1 QA (bạn), 1 Designer, 1 PM
- Sprint: 2 tuần | Release: Mỗi 4 tuần lên production
- Môi trường: Dev (localhost) → Staging (staging.shopeasy.vn) → Production (shopeasy.vn)
- CI/CD: GitHub Actions đã setup (từ bài 1-6 trong lab này)
- Sprint 5 goal: Ra mắt tính năng 'Thanh toán trả góp qua VPBank'

PHẦN A - Test Strategy Document (1.0 điểm):

Soạn tài liệu Test Strategy cho dự án ShopEasy (tối thiểu 600 từ, trình bày theo cấu trúc sau):

9. Phạm vi kiểm thử: Liệt kê 5 module IN SCOPE (đăng ký, đăng nhập, tìm kiếm, giỏ hàng, thanh toán) và 2 module OUT SCOPE. Giải thích lý do OUT SCOPE.
10. Phân loại test và tỷ lệ: Xác định % Unit / API / UI / Performance / Security. Giải thích tại sao chọn tỷ lệ đó dựa trên đặc điểm ứng dụng thương mại điện tử.
11. Tiêu chí Definition of Done: Định nghĩa rõ ràng khi nào một tính năng được coi là "đã test xong" và sẵn sàng release. Bao gồm: pass rate, coverage %, bug severity cho phép.

12. Quản lý rủi ro: Liệt kê 4 rủi ro kiểm thử thực tế (mỗi rủi ro: mô tả → xác suất → tác động → kế hoạch giảm thiểu).
13. Lịch trình kiểm thử: Bảng lịch chạy các loại test (smoke sau mỗi commit, regression hàng đêm, performance testing hàng tuần, security scan trước release).

PHẦN B - Test Plan cho Sprint 5 (1.0 điểm):

Tính năng Sprint 5: Thanh toán trả góp VPBank (3/6/12 tháng, phí 0% với đơn hàng >= 3 triệu)

14. Phân tích rủi ro nghiệp vụ: Tính năng thanh toán liên quan đến tiền thật. Liệt kê 5 kịch bản có thể gây mất tiền người dùng nếu sai.
15. Thiết kế 15 test case (điền vào bảng). Mỗi TC có: TC-ID, Tiêu đề, Loại (API/UI/Unit), Mức ưu tiên (P1/P2/P3), Bước thực hiện tóm tắt, Kết quả mong đợi:

TC-ID	Tiêu đề	Loại	Ưu tiên	Kết quả mong đợi
TC-001	Thanh toán trả góp 3 tháng thành công với đơn >= 3tr	API	P1	201, status=APPROVED
TC-002	Thanh toán trả góp bị từ chối khi đơn < 3 triệu	API	P1	400, error=ORDER_TOO_SMALL
TC-003	UI hiển thị đúng 3 tùy chọn: 3/6/12 tháng	UI	P1	3 radio button hiển thị
TC-004	...	...	...	...
TC-005 → TC-015	Sinh viên thiết kế	...	...	...

16. Xác định Blockers: Điều gì có thể ngăn bạn kiểm thử đúng hạn? (VD: Môi trường sandbox VPBank chưa ready, test data tài khoản ngân hàng chưa có)
17. Pipeline cho Sprint 5: Sửa workflow YAML để thêm bước chạy riêng testng-payment.xml sau mỗi PR nhắm vào nhánh feature/vpbank-payment.

Gợi ý soạn Test Strategy thực tế

- Câu hỏi nền tảng khi viết Test Strategy:
  - ✓ Nếu chỉ có 40 giờ test cho sprint này, bạn ưu tiên kiểm thử gì trước? Tại sao?
  - ✓ Tính năng nào có rủi ro cao nhất liên quan đến tiền, bảo mật, dữ liệu?
  - ✓ Khi nào dừng test và cho phép release? (Không phải khi 100% pass - không thực tế)
  - ✓ Làm sao QA biết developer đã sẵn sàng bàn giao để test?
- Sai lầm phổ biến trong tài liệu kiểm thử của sinh viên:

  ✗ 'Kiểm thử tất cả các tính năng' - không thực tế, không đo được

  ✗ 'Pass rate 100%' - không thực tế trong dự án lớn

  ➢ 'Smoke test pass 100%, regression pass >= 95%, không có bug P1 mở' - đo được, thực tế

Tiêu chí chấm điểm toàn bộ Lab 11

Bài 1 (1.5đ): Pipeline YAML chạy thành công trên GitHub Actions - có ảnh chụp log xanh

Bài 2 (1.0đ): Matrix Chrome + Firefox chạy song song - có ảnh chụp 2 job đồng thời

Bài 3 (1.0đ): Không có credential nào hardcode trong code - kiểm tra bằng grep

Bài 4 (2.0đ): Grid Console UI hiển thị 3 node + ảnh nhiều session chạy đồng thời + bảng so sánh thời gian

Bài 5 (1.0đ): Allure Report có biểu đồ, step-by-step, ảnh khi fail - chạy được local

Bài 6 (1.5đ): GitHub Pages URL của Allure Report hoạt động + README có badge

Bài 7 (2.0đ): Test Strategy đủ 5 phần, Test Plan đủ 15 TC có đầy đủ cột yêu cầu

HẾT

Good luck!

And good bye! Not to see you again!